



Politecnico
di Torino

Algorithms and Programming

Iniziato martedì, 21 dicembre 2021, 17:16

Stato Completato

Terminato martedì, 21 dicembre 2021, 17:16

Tempo impiegato 6 secondi

Valutazione 0,00 su un massimo di 9,00 (0%)

Domanda 1

Risposta non data

Punteggio max.:
1,00

The following recursive function is called with $i=1$, $j=1$, $n=5$, and $v=0$.

```
void rec (int i, int j, int n, int v) {  
    if (i>n) {  
        return;  
    }  
    if (j<=n) {  
        printf ("%2d ", v++);  
        rec (i, j+1, n, v);  
        return;  
    }  
    printf ("\n");  
    rec (i+1, 1, n, v);  
    return;  
}
```

Following the recursion tree, understand which is the output displayed by the function. Indicate the number of rows displayed, followed by the smaller and the higher values printed. Please, report your response as a sequence of 3 integer values separated by one single space. No other symbols must be included in the response. This is an example of the response format: 3 4 6

Risposta:



La risposta corretta è : 5 0 24

Domanda 2

Risposta non data

Punteggio max.:
4,00

A palindrome string is one that reads the same backward as well as forward. For example, the strings "aba" and "123a321" are palindromes. A sub-string of a string is a contiguous sequence of characters extracted from the original string. Thus, "3a3" is a sub-string of "123a321".

Write a **non-recursive** function called `palindrome` which receives one string `str` of unknown length and it returns the length of the longest palindrome substring of `str`. For example, if `str="1234554abccbaxxYY"` the longest palindrome substring of `str` is "abccba". Thus, the function must return 6. If there is not a palindrome sub-string in `str`, the function must return the value 0.

The function prototype is the following:

```
int palindrome (char *str);
```

Please, report the entire C code and explain (in plain English language) the logic of the code.

Domanda 3

Risposta non data

Punteggio max.:
4,00

In a linked list each node points to both the right-hand side and the left-hand side nodes. Each node stores a single string of undefined length and the two pointers. The list is accessible starting from the head pointer on the left extreme, as well as starting from the tail pointer on the right extreme. Strings are stored alphabetically ordered from left to right.

Write the function **insert** to insert a new string `str` in the list at the correct (ordered) position.

The node data structure and the function prototypes are the following:

```
typedef struct element_s {  
    char *name;  
    struct element_s *left;  
    struct element_s *right;  
} element_t;
```

```
void insert (element_t **head, element_t **tail, char *str);
```

Take particular care to: 1) manipulate empty lists, 2) manipulate the left and the right pointers of all elements involved in the insertion, and 3) dynamically allocated all required ADTs. Please, report the entire C code and explain (in plain English language) the logic of the code.
